
Generating a Kubernetes Deployment Set from a Project-Intent Domain Model

Project Task 3: Code Generation

Project Type: Code Generation

October 2, 2026

Joris Jonkers (S1829157)

Model Driven Engineering
University of Twente

Contents

1	Purpose of the Project	1
1.1	Goals	1
1.2	Role of the Code Generation	1
1.3	Refinement of Tasks 1 and 2	1
1.4	Assumptions and Conditions	2
1.5	Scope	2
2	Alternatives and Justification	2
2.1	The Template Language	2
2.2	The Structure of the Templates	3
3	Solution Design	4
3.1	Interface	4
3.2	The Shape of the Module	4
3.3	Whitespace: the Margin Rule	5
3.4	Spelling Rules	6
3.5	The Shared Index	6
3.6	What the Templates Leave to the Transformation	7
3.7	Running the Code Generation	7
4	Tests and Evaluation	7
4.1	Testing Strategy	7
4.2	Results	7
4.3	Limitations	8
4.4	Reflection	9
A	The Kinds of File and Their Templates	10
B	The Hand-in Archive	10

1 Purpose of the Project

1.1 Goals

The project generates the Kubernetes files of a small cluster from short deployment documents. Each application's repository holds one *project document*, which states what the application needs, and one *platform document* states what all applications share. Task 1 defined two metamodels in Ecore: the *source metamodel*, which describes both documents, and the *target metamodel*, which describes the *resolved model*. A resolved model records every value the platform derives for one project, and the files that will be generated from it. Task 2 implemented the model transformation from source models to a resolved model in QVT-Operational.

Task 3 implements the last step: Acceleo templates that turn a resolved model into text. The text is the *Deliverable Set*, the YAML and JSON files a cluster applies: the workloads and their service accounts, the network policies, the routes of the web proxy, the monitors, the volume claims and backups, the configuration files, the objects of the secret operator, and the policies of the secret store. The templates are run on the resolved models the Task 2 transformation writes, so the three tasks together form one pipeline from documents to files.

The code generation has two goals.

1. It writes the same files as the production implementation of the same specification, which is written in TypeScript and used to deploy the cluster, *byte for byte*. Both implementations are compared with the same committed trees of files, called *oracles*. Byte-for-byte equality is what makes the comparison meaningful: whitespace, key order and quoting decide whether two files are the same, so the templates have to decide them exactly as the production implementation's serializer does.
2. It decides nothing. Every value in a generated file is read from the resolved model. A template only chooses how a value is spelled in YAML or JSON, never which value it is, so a wrong file is always traced to the transformation or the model and never to the templates.

1.2 Role of the Code Generation

Figure 1 shows the pipeline. Xtext parses each document into a source model and OCL rejects invalid ones (Task 1). Two QVT-Operational transformations lower and resolve them into one resolved model per project (Task 2). The Acceleo module of this report writes the files of every resolved model it is given. One Java entry point, `Pipeline.render`, runs all of these steps in one pass, and the build compares the files it writes with the oracles.

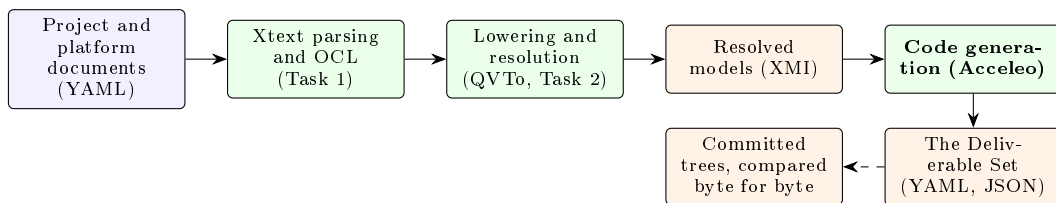


Figure 1: The Pipeline, With the Code Generation This Report Describes

1.3 Refinement of Tasks 1 and 2

Task 1 planned to write the templates against hand-written resolved models, so that the code generation could proceed while the transformation was still being written. The first template indeed ran on the hand-written model of the `minimal` example. By the time the templates were widened, the Task 2 transformation already resolved every example, so the templates were widened directly on its output, and the hand-written model is now only the expected result of the transformation for `minimal`. No test feeds a hand-written model to the templates any more.

Generating the files changed the earlier tasks in two ways:

- **The target metamodel gained the files it did not describe.** Task 1's metamodel described the workload, identity, network policy, route, monitor and secret files. The oracles also hold services, volume claims, configuration files, backup jobs, the secret operator's connection and the secret store's policies and

roles. Seven classes were added for them, one class was restructured, and a few values the files need were added, such as the endpoint of the release gate and the rule that admitted a network peer. The Task 2 report lists every change in its appendix.

- **The application revision became the production implementation's.** Every generated Canary carries the revision of its application, a digest. Task 2 first computed it over the target model's own form, which differs from the production implementation's. Byte-for-byte files need the production value, so the transformation now writes the application's element in the production implementation's JSON form and hashes that.

1.4 Assumptions and Conditions

- The resolved model is complete. The transformation stops rather than write a model with a missing value (Task 2), so a template reads every value it needs without checking for its absence, except where the target metamodel declares a value optional.
- The production implementation is the reference for the spelling of every file: its key order, its indentation, and when it quotes a string. Where YAML allows two spellings, the templates use the production one.
- Several resolved models are rendered together when their files share a directory. The routes of every project share the directory of their entry point, so the index file of that directory lists the routes of every project rendered with it.
- The templates run without Eclipse, through Acceleo's standalone API, so that the build server runs them on every change. A launch configuration runs the same module in Eclipse (Appendix B).

1.5 Scope

The templates generate every tree the production implementation generates: the share of the `minimal` project (9 files), the share of the `data` project (28 files), and the files the two share at the level of the cluster (7 files), 44 files in total. Together they use every kind of file the specification defines for a running application, apart from two.

The `auth` and `knowledge` examples also carry rendered trees, but those are written by hand as the goal state of the specification: the production implementation does not generate them yet either, so they hold no reliable oracle. They contain the two kinds of file no implementation generates yet, a disruption budget and a database migration job, and the target metamodel does not describe those two kinds. Section 4.3 discusses this.

2 Alternatives and Justification

2.1 The Template Language

The course asks for Acceleo, and Acceleo exists in two incompatible generations. Table 1 compares them with the two languages a generator of this kind would otherwise be written in.

Language	For	Against
Acceleo 3 (MTL)	Implements the OMG MOF Model-to-Text standard; the most documented generation.	The older generation, kept for compatibility; its generator project assumes an Eclipse workspace, and it is not the Acceleo of the release the build pins.
Acceleo 4	Expressions in AQL, the query language the Sirius tools share; a maintained standalone API; released in the same Eclipse release train as the Xtext, OCL and QVTo versions the build already pins.	A newer, thinner documentation; its whitespace rule (Section 3.3) differs from every other template language.
Xtend	Typed template expressions in Java's type system.	Not a model-to-text language the course accepts; a template is Java code in disguise.
Java and a YAML library	The production implementation's own approach: build typed objects, let one serializer write the bytes.	Not model-to-text: a serializer owns the formatting, so the templates would decide nothing about the text, which is what the task asks for.

Table 1: Template Languages Considered

Acceleo 4 was chosen. The deciding fact is the release train: the build pins one Eclipse release for every modelling tool (Task 1), and the Acceleo of that release is Acceleo 4. Its standalone API runs the templates from a JUnit test or the command-line pipeline with no workspace. Its expression language, AQL, is close enough to OCL that the queries read like the constraints of Task 1.

2.2 The Structure of the Templates

Table 2 summarises the four decisions about how the templates are organised. The subsections give the reasons.

Decision	Chosen	Rejected
Who formats the text Modules	The templates, character for character One module, one template per kind of file, shared fragments	A YAML serializer after the templates One module per kind of file
Files that several projects share	One query over every model rendered together	Merging the models in Java first, or a second transformation
Where spelling rules live	AQL queries in the module	Attributes the transformation computes

Table 2: Structural Decisions

2.2.1 The Templates Write Every Character

The production implementation builds typed objects and lets one YAML serializer write them, so its key order, indentation and quoting are the serializer's. The Acceleo implementation could have generated a rough text and run the same kind of serializer over it. That was rejected for two reasons. First, it would move the formatting out of the model-to-text transformation, which is what this task is meant to show. Second, it would weaken the comparison: a serializer that normalises the output would make two different texts equal, and the oracle would no longer prove that the templates are right. The templates therefore write every character, and the test compares their output with the oracle with nothing in between.

2.2.2 One Module

Every Kubernetes object carries the same metadata block and the same fixed set of five labels, and several objects share larger fragments: the label set, a network peer, a probe target, an environment list. With one module per kind of file, each fragment would be written several times or imported across modules. With one module, each fragment is one template, and the sixteen kinds of file each have one template that spells their format. The module is long (831 lines), but it is read top to bottom like the list of generated files.

2.2.3 Files Several Projects Share

Each entry point of the web proxy has one directory, and its index file lists the routes of every project the entry point serves. The transformation resolves one project at a time (Task 2), so each resolved model only knows its own routes. Three ways to write the shared index were considered. A Java step could merge the index entries of all models before the templates run, but that is a post-processing step between the transformation and the templates. A second transformation could combine the resolved models into one model of the whole cluster, but that adds a metamodel for a single file. The chosen way is a query: when several resolved models are rendered together, the index template lists the routes that every model puts under the same path. Each model then writes the same complete file, so the order in which the models are rendered does not matter.

2.2.4 Spelling Rules as Queries

Some values are spelled differently in YAML than in the model: a duration 60s is written as the integer 60, a precedence of 1 is written as the proxy priority 999, and a string such as "50" must be quoted, or a YAML reader takes it for a number. The transformation could compute these spellings as extra attributes, but then the resolved model would contain YAML details, which the target metamodel keeps out on purpose. They are AQL queries in the module instead, one per rule, so every rule about the text lives in the code generation.

3 Solution Design

3.1 Interface

The code generation is one Acceleo module, `render.mtl`, in the `render` bundle, and one Java class, `Rendering`, that runs it. The module's input is the target metamodel: its main template takes one `ResolvedDeployment`. Its output is a tree of files under a root directory, each at the path the resolved model assigns it. The module reads nothing else: no file, no environment, no clock.

`Rendering.render` takes a list of resolved models and a root directory. It copies the models into one resource, so the caller's models are left as they were, and so a query can see every model rendered together (Section 3.5). It then loads the module through Acceleo's standalone API and generates. Any diagnostic Acceleo reports, whether a parse error in the module or an evaluation error in a template, fails the run with Acceleo's own message, so a half-written tree is never mistaken for a whole one. Listing 1 shows the core of the runner.

```
ResourceSet resources = new ResourceSetImpl();
Resource model = new XMIRResourceImpl(URI.createURI(MODEL));
resources.getResources().add(model);
model.getContents().addAll(EcoreUtil.copyAll(deployments));
ClassLoaderQualifiedNameResolver resolver = new ClassLoaderQualifiedNameResolver(
    Rendering.class.getClassLoader(), resources.getPackageRegistry(), "::");
IQualifiedNameQueryEnvironment environment =
    AcceleoUtil.newAcceleoQueryEnvironment(Map.of(), resolver, resources, false);
// A template's parameter type resolves only against the packages its environment knows.
environment.registerEPackage(ResolvedDeploymentPackage.eINSTANCE);
AcceleoEvaluator evaluator = new AcceleoEvaluator(environment.getLookupEngine(), "\n");
resolver.addLoader(new ModuleLoader(new AcceleoParser(), evaluator));
AcceleoUtil.generate(evaluator, environment, (Module) resolver.resolve("render", model,
    new DefaultGenerationStrategy(resources.getURIConverter(), new DefaultWriterFactory()),
    URI.createFileURI(root.toAbsolutePath() + "/"), null, new BasicMonitor()));
if (evaluator.getGenerationResult().getDiagnostic().getSeverity() != Diagnostic.OK)
    throw new IllegalStateException("the rendering did not complete: " + ...);
```

Example 1: The Runner, Abridged

The pipeline's entry point, `Pipeline.render`, puts the three tasks in one call: it parses and checks every document, lowers and resolves each project named, and hands the resolved models to `Rendering.render`. The build calls it for the union of the minimal and data projects and leaves the tree under `bundles/cli/target/parity/rendered/`, where the parity suite compares it with the oracles (Section 4.1).

3.2 The Shape of the Module

The module has 35 templates and 16 queries. Figure 2 shows how they call each other.

- **The main template** walks the resolved model's generated-file entries, the *deliverables*, and opens one file per entry at its path (Listing 2).
- **document** writes the one header line every YAML file carries, or writes nothing before a JSON document, which cannot carry a comment. `yaml` then dispatches on the kind of file.
- **One objects template per kind of file** spells that file's format: sixteen kinds, from a namespace to a backup job. Appendix A lists them, with the objects each writes.
- **Fragments** are what several objects share: the metadata block, the fixed label set, a network peer, a probe target, a container, an environment list. Each is written once, unindented, and every caller places it at its own depth (Section 3.3).
- **Queries** hold every rule about spelling: durations as seconds, precedences as priorities, quoting, the name of a volume, the shared index (Section 3.4).

```
[comment @main /]
[template public render(deployment : resolveddeployment::ResolvedDeployment)]
  [for (d : resolveddeployment::Deliverable | deployment.deliverables)]
    [file (d.path, overwrite, 'UTF-8')]
    [d.document()/]
  [/file]
[/for]
[/template]

[template private document(d : resolveddeployment::Deliverable)]
  [if (d.ocllsKindOf(resolveddeployment::VaultPolicyFile))]
    [d.ocllsType(resolveddeployment::VaultPolicyFile).json()/]
  [elseif (d.ocllsKindOf(resolveddeployment::VaultRoleFile))]
    [d.ocllsType(resolveddeployment::VaultRoleFile).json()/]
```

```

[else]
  # GENERATED. Never hand-edit.
  [d.yaml()/]
[/if]
[/template]

```

Example 2: The Main Template and the Header

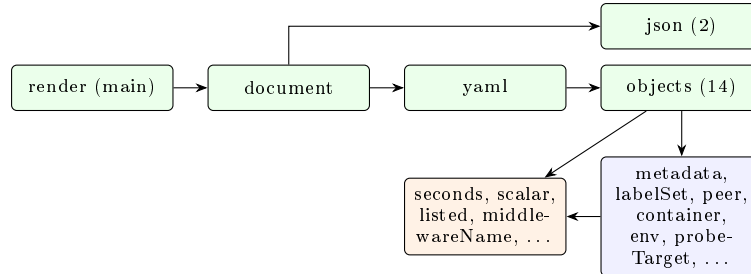


Figure 2: How the Templates and Queries Call Each Other

3.3 Whitespace: the Margin Rule

YAML encodes structure in indentation, so the whitespace a template writes is part of the result. Acceleo 4 has two rules about it, and the module's structure follows from them.

1. **Every line of a block's body carries two spaces of margin, which are not written.** The body of a template, a `for` or an `if` is indented two spaces deeper than its header, and those two spaces are removed from the output. A line with less margin is a parse error.
2. **A call written at a column repeats that column's text before every line the call writes.** If a template is called after six spaces, the six spaces are written before every line of its output.

The second rule is what makes fragments possible. The label set is one template that writes five lines starting at column zero (Listing 3). The metadata block calls it one level under its own `labels:` key, and the workload's pod template calls it several levels deeper. The same five lines therefore appear at every depth YAML needs without being written twice.

```

[template private metadata(p : resolveddeployment::ResolvedProcess)]
  name: [p.name/]
  namespace: [p.application().namespace/]
  labels:
    [p.labels()/]
[/template]

[template private labelSet(name : String, component : String, application : String)]
  app.kubernetes.io/name: [name/]
  app.kubernetes.io/instance: [name/]
  app.kubernetes.io/part-of: [application/]
  app.kubernetes.io/managed-by: deploy-kit
  app.kubernetes.io/component: [component/]
[/template]

```

Example 3: A Fragment, Placed at Its Caller's Depth

The same rule has one trap. The repeated text is copied literally, so a call written after `-` would start *every* line with a dash, and turn each key of a container into a list item of its own. A fragment that is a list item therefore opens with its own `-` and indents the rest of its lines two spaces, and its caller places it with spaces only. The container fragment of Listing 4 is written that way. Conditional lines follow the first rule: an `if` adds a level of margin, so a line inside it is indented two spaces more in the module than in the output, as the `fsGroup` line shows.

```

  securityContext:
    runAsNonRoot: true
    runAsUser: [p.uid/]
    runAsGroup: [p.gid/]
  [comment A volume is written as the image's group, and only a volume is. ./]
  [if (p.volumes->notEmpty())]
    fsGroup: [p.gid/]

```

```

[/if]
    seccompProfile:
      type: RuntimeDefault
    containers:
      [p.container()/]

[template private container(p : resolveddeployment::ResolvedProcess)]
- name: [p.name/]
  image: [p.image/]
  [if (p.surfaces->notEmpty())]
    ports:
      [for (s : resolveddeployment::ResolvedSurface | p.surfaces)]
        - name: [s.name/]
          containerPort: [s.port/]
      [/for]
  [/if]
[/if]

```

Example 4: Part of the Workload Template, and the Container Fragment

3.4 Spelling Rules

Every rule about how a value is written is one query, so a template never computes anything itself.

- **Durations.** The model records a duration as text, such as 60s; Kubernetes fields take whole seconds. `seconds` converts seconds, minutes and hours.
- **Quoting.** A string is written plain unless the YAML 1.2 core schema would read it as null, a boolean, an integer or a float, in which case it is double-quoted, as the production implementation's serializer does (Listing 5). The environment value 50 is therefore written "50", and `production` is written plain.
- **Block scalars.** A configuration file's text is written as a literal block (`|`), each line indented under its key, an empty line left empty, and its last line break kept the way the serializer keeps it.
- **Names.** A volume for a writable path is named after the path, and a file mounted from a configuration file is named after the last segment of its source.
- **Priorities and middleware.** A route's precedence becomes a proxy priority, 1000 minus the precedence, and a middleware step becomes the name of the middleware object the edge project renders.

```

[query private seconds(duration : String) : Integer =
  let amount : Integer = duration.substring(1, duration.size() - 1).toInteger() in
  if duration.endsWith('h') then amount * 3600
  else if duration.endsWith('m') then amount * 60 else amount endif endif /]

[query private scalar(value : String) : String =
  if value.matches('~|null|Null|NULL|true|True|TRUE|false|False|FALSE|[-]?[0-9]+|0o[0-7]+|0x[0-9a-fA-F]
  +|[-+]?([.][0-9]+|[-+]?[0-9]*)?|([eE][-+]?[0-9]+)?|[-+]?[.](inf|Inf|INF)|[.](nan|NaN|NAN)')
  then '"' + value + '"' else value endif /]

```

Example 5: Two Spelling Queries

3.5 The Shared Index

The index of an entry point's directory lists the routes of every project the entry point serves (Section 2.2.3). Its template does not read its own entry's list but calls `listed` (Listing 6), which collects the entries of every index file at the same path in every resolved model rendered together, without duplicates, in name order. Each model then writes the same file, so writing it twice is harmless and the order of the models does not matter. A first version read the other models through the resource that holds them; AQL cannot navigate a resource, because a resource is not a model element, so the query uses `allInstances` instead, which the query environment answers over the resource set it was given.

```

[template private objects(d : resolveddeployment::IndexFile)]
---
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  [for (r : String | d.listed())]
    - [r/]
  [/for]
[/template]

[query private listed(d : resolveddeployment::IndexFile) : Sequence(String) =
  resolveddeployment::IndexFile.allInstances()->select(i | i.path = d.path).resources
  ->asOrderedSet()->sortedBy(r | r) /]

```

3.6 What the Templates Leave to the Transformation

Because the templates decide nothing, several decisions that look like formatting are made by the transformation and only read here.

- **Which files exist.** The transformation plans a file only when it would hold at least one object, and assigns its path. A template never skips a file or invents a path.
- **What each index lists.** The transformation lists, for every directory, the files a cluster applies from it, leaving out the network policies and the secret store's documents, which nothing applies yet.
- **Which monitor a process gets.** A blue-green process is scraped through its pods and any other through its service; the transformation records the kind, and the template writes `PodMonitor` or `ServiceMonitor` from it.
- **Who holds a secret.** The transformation lists, per application, every identity that synchronises a secret: each process, and the backup identity that holds the backup's credential.

3.7 Running the Code Generation

The build runs the templates headless: `./mvnw clean verify` in `emf/` runs the render bundle's own tests, then the pipeline, which writes the rendered tree, then the parity suite, which compares it. In Eclipse Modeling Tools with the Acceleo 4 SDK, `render.launch` in the render bundle runs the same module on the resolved model of `minimal` that the build leaves in the parity output, and writes its files under the bundle's `target/render/`.

4 Tests and Evaluation

4.1 Testing Strategy

The code generation is tested at three levels, and every level runs in the build on every change.

1. **The oracle comparison.** The build runs the whole pipeline on the union of the `minimal` and `data` projects and compares the tree it writes with the committed trees, file for file and byte for byte. The comparison sits in a separate test module that holds no EMF type and calls no Java code: it reads only the files the pipeline left behind, the same interface the production implementation's own test reads. A file the run should have written and did not fails as a missing file.
2. **The pipeline's entry point.** JUnit tests call `Pipeline.render` for each project's share and for the shared files, render the same inputs twice into two directories and compare them, and check that a set the constraints refuse writes no file at all.
3. **The runner.** The render bundle's own tests run the module on a resolved model built in code, check the one file it writes, check that the caller's model is left as it was, and run a deliberately broken module to check that a parse error fails the run with Acceleo's message.

The Java code of the render bundle is held to 100% line and branch coverage and a 100% mutation score with PIT, like every module of the implementation. Mutation testing does not reach the templates themselves, so their evidence is the oracle comparison: a template that writes one character differently fails it.

4.2 Results

Table 3 shows the result. Every file of every tree the production implementation generates is equal, and the second render of the same inputs is byte-identical to the first.

The 44 files exercise every one of the sixteen kinds of file: a blue-green workload with its Canary and pod monitor in `minimal`, and in `data` three stateful services with their services, volume claims, backup jobs, a configuration file, a sidecar container, environment variables read from synchronised secrets, the secret operator's connection and objects, and the secret store's policies and roles.

Committed tree	Files	Result
minimal/rendered/	9	equal byte for byte
data/rendered/	28	equal byte for byte
_estate/rendered/	7	equal byte for byte
Total	44	every file equal; no extra file written

Table 3: The Rendered Trees Compared With the Oracles

4.2.1 Loading the Generated Files

The task asks for the generated code to be loaded in a tool that handles it. For Kubernetes files there are two such tools short of a live cluster, and the tree passes both.

- **kustomize**, which Flux uses to apply a directory, builds every directory from its index file: the **notes** project to 5 objects, the **data** project to 27, and the shared directory of the public entry point to 2. A wrong index entry, a missing file or a file that is not valid YAML fails the build.
- **kubeconform** validates every object against the schemas of Kubernetes 1.31 and, for the custom resources of Flagger, the Prometheus operator, Traefik and the Vault secrets operator, against the schemas in the public CRD catalogue: all 47 objects in the 40 YAML files are valid, with unknown fields refused (**-strict**).

The same kubeconform check is a gate of the repository, run on the committed trees, so the oracles themselves are known to be valid. Because the generated tree equals them byte for byte, it inherits that validity; the run above checks the generated tree directly as well.

4.2.2 Defects Found by Testing

The oracle comparison found every defect below. None of them would have been visible by reading the generated files, because each produced files that looked plausible.

- **No file at all.** The first run generated nothing and reported no error. The main template’s parameter type resolves only against the packages the query environment knows, and the target metamodel had not been registered with it, so no model element matched the main template. The runner now registers the package, and the tests check the files, not only the diagnostic.
- **The margin rule.** The first version of the module was written as a plain text template and failed to parse, because every body line lacked its two spaces of margin (Section 3.3).
- **A dash on every line.** A container called after `-` turned each of its keys into a separate list item, and so did a route, because the text before a call is repeated on every line it writes.
- **Nested indentation.** The environment list, the backup’s service account and the synchronised secrets came out two columns too deep: a block nested in a **for** and an **if** has two levels of margin, not one.
- **A resource is not a model element.** The first shared-index query navigated the resource holding the models, which AQL cannot do; it now uses **allInstances** (Section 3.5).
- **Two defects in the transformation.** Comparing the **data** tree showed that the transformation chose a stateful controller for a process with a volume, where the production implementation and the oracle use a deployment, and comparing the Canaries showed that the revision differed from the production implementation’s (Section 1.3). Both were fixed in the transformation, not in the templates.

4.3 Limitations

- **Two kinds of file are not generated.** A disruption budget and a database migration job appear only in the hand-written goal-state trees of **auth** and **knowledge**. Neither implementation generates them yet, and the target metamodel does not describe them.
- **The delivery machinery is not generated.** The processes that run the delivery itself (the proxies, Flagger and the release gate) roll out in a third way the specification defines, which neither implementation renders yet, so the projects holding them have no rendered tree.
- **The quoting rule is partial.** The production serializer also quotes a string that starts with an indicator character or holds a colon followed by a space. No value in the examples needs that, so the templates do not spell it; such a value would be written unquoted and would differ from the production file.

- **Some template branches are not exercised by an example.** No example has a route that matches an exact path, a duration given in minutes or hours, a configuration file without a final line break, or a boolean-looking environment value. These branches are written to the same rules as the exercised ones, but no oracle proves them.

4.4 Reflection

Comparing byte for byte made the task harder and the result more convincing. Most of the work was not in choosing which objects to write, which the target metamodel already decides, but in matching the production serializer's text exactly: its key order, its quoting and its indentation. That precision is also what made the comparison find real defects in the transformation, which a looser comparison would have hidden.

Keeping the templates free of decisions worked as intended. Every time a template seemed to need a decision, such as which monitor to write or which identities hold a secret, the decision moved into the transformation and the target metamodel gained the value, so the templates stayed a pure spelling of the model. The cost was a larger target metamodel than Task 1 planned.

Acceleo 4's margin rule was the steepest part to learn, because it differs from every other template language: indentation in the module is both syntax and output. Once understood, it was also what made the fragments work, because one template can be placed at any depth by its caller.

A The Kinds of File and Their Templates

Table 4 lists the sixteen kinds of file the target metamodel describes, the template that writes each, and the objects in it. Every file but the two Vault documents starts with the header line and holds one or more YAML documents, each after its own `--` separator.

Kind (class)	File	Objects written
NamespaceFile	<code>namespace.yaml</code>	The project's namespace.
IndexFile	<code>kustomization.yaml</code>	The index of a directory: every file and subdirectory a cluster applies from it; a shared index lists every model's entries (Section 3.5).
NetworkPolicyFile	<code>networkpolicy.yaml</code>	The project's default-deny policy, or one policy per process with its admitted peers.
WorkloadFile	<code>workload.yaml</code>	One deployment per process: its strategy, security context, containers, sidecars, probes, mounts and volumes.
SwitchoverFile	<code>canary.yaml</code>	One Flagger Canary per blue-green process, its webhooks calling the release gate.
ServiceFile	<code>service.yaml</code>	One service per other process that serves a port.
IdentityFile	<code>serviceaccount.yaml</code>	Each process's service account, and its backup identity's.
ClaimFile	<code>pvc.yaml</code>	A claim per volume, and per backed-up volume the claim its copies land on, kept from pruning.
ConfigMapFile	<code>configmap.yaml</code>	An immutable configuration map per configuration file, its text as a block scalar.
BackupFile	<code>backup.yaml</code>	A scheduled backup job per backed-up volume.
MonitorFile	<code>podmonitor.yaml</code> or <code>servicemonitor.yaml</code>	The monitor of a scraped application.
SecretFile	<code>vso.yaml</code>	Per identity holding a secret: how the operator authenticates as it, and one synchronised secret per grant.
ConnectionFile	<code>vaultconnection.yaml</code>	The secret operator's connection to the secret store.
IngressRouteFile	<code><application>-<exposure>.yaml</code>	A Traefik route per exposure, in its entry point's shared directory.
VaultPolicyFile	<code><identity>.policy.json</code>	The Vault policy an identity's grants derive, as JSON.
VaultRoleFile	<code><identity>.role.json</code>	The Kubernetes authentication role that binds the identity to its policy, as JSON.

Table 4: The Kinds of File

B The Hand-in Archive

The archive holds this report as `report.pdf`, a `README.md` with the steps below, and two folders, laid out as Table 5 shows. The first, `task-3-code-generation/`, is an Eclipse project for reading the transformation and its results, imported with File > Import > Existing Projects into Workspace. The second, `implementation/`, contains the EMF implementation and the examples its tests read.

Every resolved model declares its metamodel in `xsi:schemaLocation`, so Eclipse opens it in the Sample Reflective Ecore Model Editor without a registered package.

`./mvnw clean verify` in `implementation/emf` builds the implementation and runs every test; it needs JDK 21. It parses every example, runs both transformations and the templates, writes the rendered tree under `bundles/cli/target/parity/rendered/`, and runs the parity test that compares it with the committed trees. To run the templates in Eclipse Modeling Tools with the Acceleo 4 SDK, import `implementation/emf` as existing Maven projects after that build, and run `render.launch` in `dev.jorisjonkers.deploykit.emf.render`. It renders `minimal` from the resolved model the build wrote.

Path	Contents
<code>model/resolved-deployment.ecore</code>	The target metamodel, the templates' input.
<code>templates/render.mtl</code>	The Acceleo module.
<code>examples/<example>/<project>.resolveddeployment</code>	The resolved model of <code>minimal</code> and of <code>data</code> the transformation wrote, the templates' source models.
<code>generated/</code>	The tree the templates wrote from both, as the build left it.
<code>expected/</code>	The committed trees it equals: <code>minimal/</code> , <code>data/</code> and <code>_estate/</code> .
<code>evidence/</code>	The output of <code>kustomize</code> build for each directory and of <code>kubeconform</code> for the tree.

Table 5: The Project of the Archive